



Kéo đến trang này để mở note riêng

Từ Chatbot Đến Agentic Agent

AICB-P1 · Ngày 3 · Design Pattern ReAct

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 17/03/2026

HÃY SUY NGHĨ...

*“ChatGPT là chatbot hay agent?
Siri thì sao? Cursor IDE thì sao?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học



1. 3 Kiểu Hệ Thống AI
2. Agentic Fit Framework
3. Kiến Trúc Agent
4. ReAct Pattern
5. Agent Loop: Code Anatomy
6. Live Demo & Debug
7. Chatbot vs Agent
8. Lab 3

- Phân biệt được **rule-based bot**, **LLM chatbot**, và **agent**
- Dùng **Agentic Fit** để biết khi nào nên nâng từ chatbot lên agent
- Hiểu và giải thích được vòng lặp **ReAct: Thought → Action → Observation**
- Build được **ReAct agent đầu tiên** với tools, system prompt, và safeguard cơ bản

Deliverable Cuối Ngày

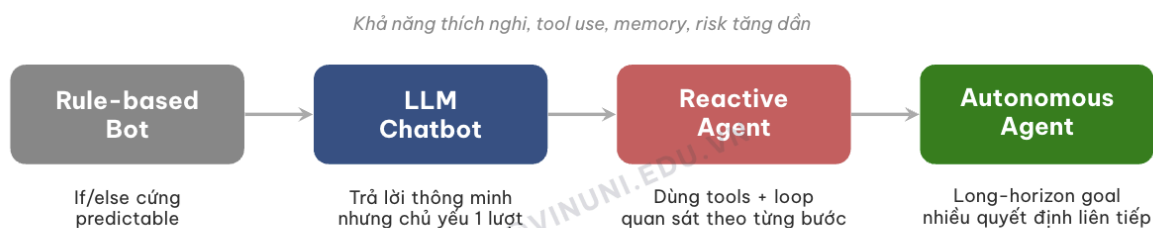
Chatbot baseline + ReAct agent cho cùng một bài toán, kèm trace và flowchart luồng xử lý

- 5 test cases để so sánh chatbot và agent
- 1 trace Thought / Action / Observation của agent
- 1 nhận định rõ: khi nào chatbot đủ, khi nào agent vượt trội

3 Kiểu Hệ Thống AI

Từ bot có rule đến agent có khả năng lập kế hoạch và dùng công cụ

Spectrum: Bot → Chatbot → Agent



Không phải mọi thứ dùng LLM đều là agent. Agent chỉ xuất hiện khi hệ thống phải quyết định, hành động, quan sát kết quả, rồi lặp lại.

Tiêu chí	Rule-based Bot	LLM Chatbot	Agent
Cách xử lý	If/else cố định	Sinh câu trả lời tốt theo context	Plan → act → observe → adapt
Flexibility	Thấp	Trung bình	Cao
Memory	Gần như không có	Ngắn hạn trong context	Ngắn hạn + có thể thêm long-term memory
Tool use	Hard-coded	Có thể gọi tool theo chỉ định	Chủ động chọn tool theo bước tiếp theo
Cost	Thấp nhất	Trung bình	Cao hơn do loop và nhiều calls
Risk	Logic dễ kiểm soát	Hallucination / format drift	Hallucination + tool misuse + loop
Ví dụ phù hợp	Menu IVR, form validation	FAQ, support cơ bản	Booking, research, coding assistant

So sánh trực quan để chọn đúng mức độ phức tạp

Ví Dụ Nhanh: Cùng Một Câu Hỏi, 3 Mức Độ Hệ Thống

Bài toán: “Tìm vé HAN → HCM dưới 2 triệu, rồi gợi ý mang gì nếu trời mưa.”

Bot có rule

- Trả menu lựa chọn cố định
- Không search được dữ liệu mới
- Không tổng hợp nhiều điều kiện

LLM chatbot

- Viết câu trả lời mượt
- Nhưng không tự truy vấn giá vé thật

Reactive agent

- Tách goal thành 2 việc: tìm vé + check thời tiết
- Gọi từng tool theo bước
- So sánh kết quả rồi trả lời gộp

Lưu ý: Nếu bài toán không cần dữ liệu mới, nhiều bước, hay quyết định động, agent thường là overkill.

Agentic Fit Framework

4 tiêu chí để biết bài toán có thật sự cần agent hay không

4 Tiêu Chí Agentic Fit

1. Multi-step Reasoning

Bài toán có cần chia thành nhiều bước phụ thuộc nhau không?

3. Dynamic Decision

Mỗi bước tiếp theo có phụ thuộc vào kết quả vừa quan sát không?

2. Tool Interaction

Hệ thống có cần gọi search, API, database, calculator, browser, file system...?

4. Long Horizon

Hệ thống có phải giữ mục tiêu xuyên suốt qua nhiều vòng lặp hoặc nhiều state không?

Nếu đa số tiêu chí chỉ ở mức 1-2/5, hãy bắt đầu bằng chatbot hoặc workflow đơn giản.

Scoring Matrix: Có Cần Agent Không?

Use case	Reasoning	Tool use	Dynamic decision	Tổng
FAQ nội bộ HR	1	1	1	3
Tóm tắt hợp đồng và highlight risk	3	2	2	7
Booking assistant du lịch	4	5	4	13
Research agent tìm đối thủ cạnh tranh	4	4	4	12
Code assistant có test & fix loop	5	5	4	14

Gợi ý đọc điểm: 0-5 = chatbot/rule đủ 6-10 = augmented chatbot 11+ = agent đáng thử

Chấm nhanh theo thang 1-5 cho từng tiêu chí

- ☐ **Bài toán 1 bước:** hỏi đáp, tra FAQ, phân loại cơ bản
- ☐ **Không có tool nào để gọi:** agent chỉ “suy nghĩ” nhưng không hành động được
- ☐ **Mọi thứ phải 100% deterministic:** mỗi sai sót đều rất đắt
- ☐ **Chi phí latency không chấp nhận được:** loop 3–5 bước là đã quá chậm
- ☒ **Nguyên tắc:** luôn benchmark rule / workflow / chatbot trước khi mở agent loop

Case Study: Chatbot Đủ Hay Cần Agent?

Customer FAQ

- Câu hỏi lặp lại, intent khá ổn định
- Chủ yếu retrieve policy rồi trả lời
- Có thể thêm RAG nhưng chưa cần autonomy
- **Best fit:** chatbot có retrieval

Booking Assistant

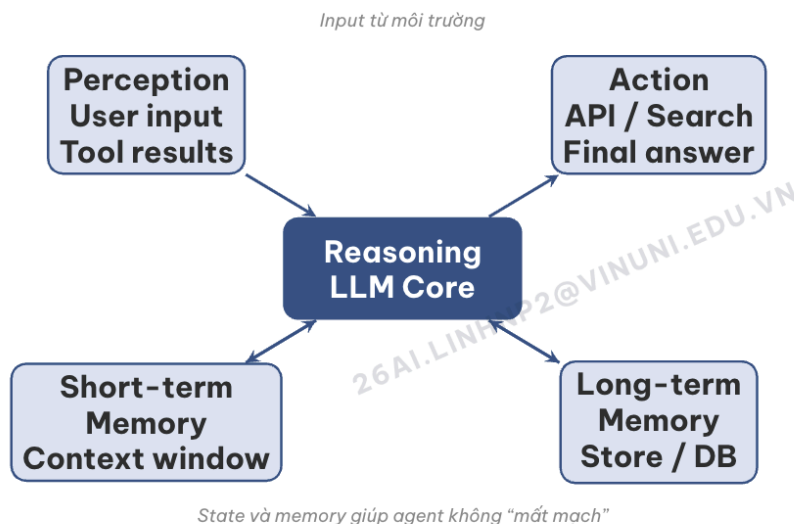
- Nhiều ràng buộc: thời gian, ngân sách, preference
- Phải search, so sánh, hỏi lại, rồi chốt phương án
- Bước sau phụ thuộc kết quả bước trước
- **Best fit:** reactive agent có tool use



Bắt đầu từ cấu trúc đơn giản nhất đủ dùng. Agent là pattern mạnh nhưng cũng đắt nhất về cost, eval, guardrails, và vận hành.

Kiến Trúc Agent

Perception, reasoning, action, memory và luồng thông tin giữa các khối



Kéo đến trang này để mở note riêng

- **Perception:** agent nhận text, tool output, feedback
- **Reasoning:** phân tích trạng thái và chọn bước tiếp theo
- **Action:** gọi tool hoặc trả lời user
- **Memory:** giữ goal, facts, và intermediate results

4 khối kiến trúc thường kéo theo 4 nhóm cost chính: token, storage, API, và latency.

Memory: Short-term vs Long-term

Short-term memory

- Nằm trong context window
- Dùng cho task hiện tại
- Rẻ để implement, nhưng dễ đầy

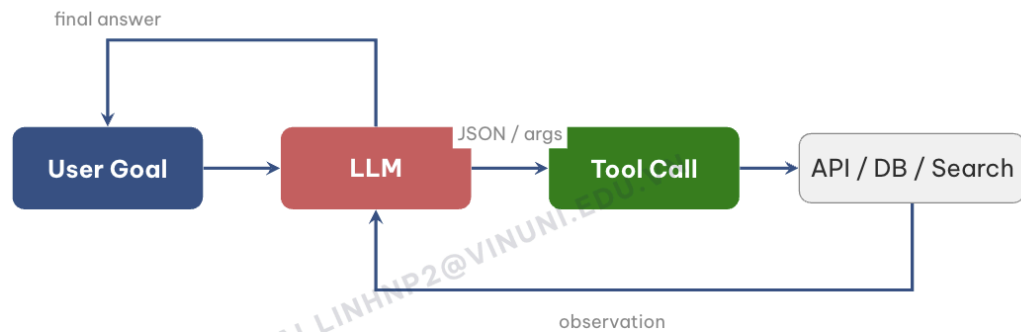
Phù hợp khi

- Cuộc hội thoại ngắn
- Goal chỉ kéo dài vài bước

Long-term memory

- Lưu facts, preferences, hay state ngoài context
- Có thể là DB, vector store, key-value store
- Cần retrieval strategy và permission model

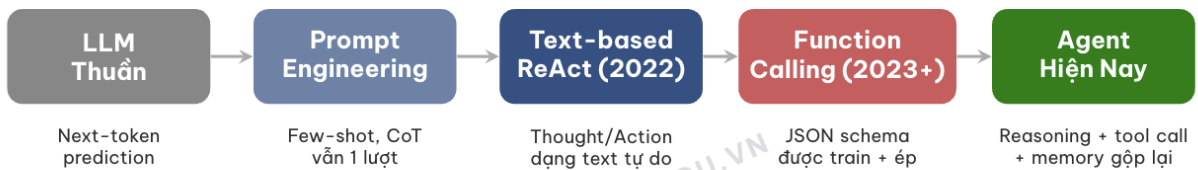
Lưu ý: Không phải thêm memory là agent giỏi hơn. Memory chỉ có ích khi chiến lược đọc/ghi và quyền truy cập được thiết kế rõ.



- Tool definitions phải rõ input / output / error mode
- Agent mạnh lên nhờ tool, nhưng cũng dễ fail hơn vì external dependency
- Tool calling là cầu nối giữa reasoning trong model và hành động ngoài thế giới thực

ReAct Pattern

Reasoning + Acting: cách đơn giản nhất để biến LLM thành agent có thể debug được



Hiểu ReAct dạng text (bước 3) giúp thấy rõ vấn đề nó gặp phải, để hiểu tại sao ngành chuyển sang function calling (bước 4). Function calling không “thay thế” ReAct – vòng lặp Thought/Action/Observation vẫn còn đó, chỉ là Action giờ đáng tin cậy hơn nhiều.

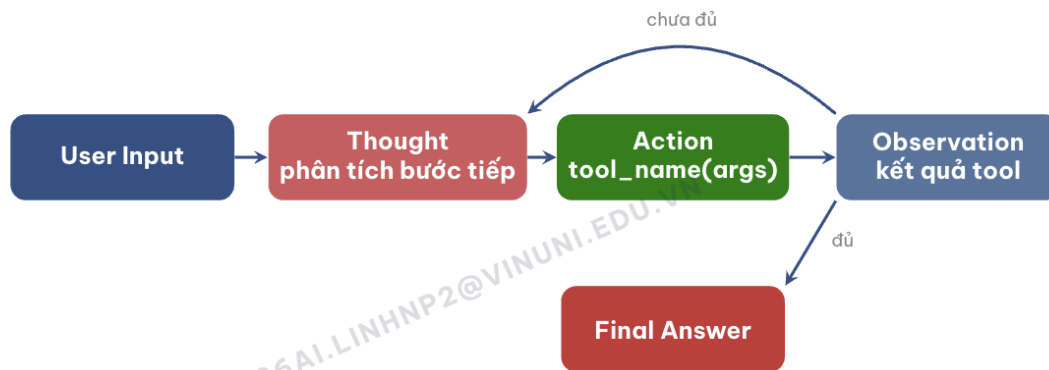
Định Nghĩa

ReAct = Reasoning + Acting

ReAct là pattern kết hợp **suy luận theo từng bước** với **gọi công cụ và quan sát kết quả**. Thay vì trả lời ngay, agent sẽ lặp qua các bước:

- **Thought:** mình đang thiếu gì, nên làm gì tiếp?
- **Action:** gọi tool nào, với tham số nào?
- **Observation:** kết quả trả về là gì?
- Lặp lại đến khi đủ thông tin để trả lời hoặc gặp điều kiện dừng

ReAct Loop: Thought → Action → Observation



ReAct mạnh vì trace lý do hành động được bộc lộ ra ngoài, giúp con người debug và can thiệp dễ hơn so với chỉ nhìn final answer.

Trace Ví Dụ: Tìm Chuyến Bay HAN → HCM (1/2)

Thought 1: Tôi cần tìm chuyến bay sáng mai từ HAN tới HCM dưới 2 triệu.

Action 1: `search_flights(origin="HAN", destination="SGN", date="2026-03-18", max_price=2000000)`

Observation 1: Có 2 lựa chọn phù hợp: VietJet 06:10 giá 1.75M, Vietnam Airlines 08:20 giá 1.95M.

Thought 2: User cũng hỏi về trang phục nếu trời mưa. Tôi cần check thời tiết tại HCM.

Action 2: `get_weather(city="Ho Chi Minh City", date="2026-03-18")`

Observation 2: Nhiệt độ 27–32C, xác suất mưa 70%.

Thought 3: Tôi đã có cả lựa chọn chuyến bay và dữ liệu thời tiết. Có thể kết luận.

Final Answer: Gợi ý chuyển 06:10 giá 1.75M; mang áo mỏng, giày dễ khô, ô gấp hoặc áo mưa nhẹ.

Điểm mạnh không chỉ là answer tốt hơn, mà là con người nhìn được agent đã hành động dựa trên quan sát nào.

ReAct Tốt Ở Điểm Nào?

Ưu điểm

- Dễ đọc trace và debug
- Tự quyết được bước tiếp theo từ observation
- Phù hợp các bài toán search / booking / investigation / coding
- Có thể cài safeguard ở từng vòng lặp

Giới hạn

- Tốn nhiều token và latency hơn chatbot
- Dễ loop hoặc gọi sai tool
- Cần eval theo trace, không chỉ final answer
- Không phù hợp bài toán đơn giản hoặc cần deterministic tuyệt đối

Lưu ý: ReAct dễ bắt đầu nhất, nhưng khi hệ thống nhiều nhánh hơn, nên chuyển sang graph/state machine rõ ràng.

Từ ReAct Đến Function Calling

Vì sao text-based ReAct gặp giới hạn, và cách function calling giải quyết bằng schema được train sẵn

Hạn Chế Cốt Lõi Của Text-based ReAct



Trong paper gốc, Action chỉ là **một chuỗi text** theo format cố định, ví dụ `search[entity]` hay `lookup[string]` – mô hình phải tự “nhớ” đúng cú pháp, và hệ thống bên ngoài phải **parse bằng regex/string matching** để biết nên gọi tool nào với tham số gì.

- Không có **schema** nào để model hay hệ thống kiểm tra tính hợp lệ trước khi gọi
- Chỉ cần thiếu dấu hai chấm, sai hoa/thường, hoặc quên “Action Input:” là parser vỡ – lỗi thường gặp: “*Could not parse LLM output*”
- Model có thể lặp lại Thought/Action cũ mà không tiến thêm (chính paper ReAct cũng ghi nhận vấn đề này)
- Format dễ trôi (drift) khi đổi model, đổi phiên bản, hoặc đổi prompt – rất khó bảo trì ở production

Lưu ý: Gốc rễ vấn đề: hệ thống chỉ prompt model trả lời đúng *định dạng text*, chứ chưa từng gửi cho model một **hợp đồng máy đọc được (schema)** để bám

Model output (text tự do):

Thought: Tôi cần tìm chuyến bay HAN→SGN.
action: search_flights("HAN", "SGN")

Parser (regex chỉ khớp “Action:” viết hoa, đúng format):

ValueError: Could not parse LLM output: missing 'Action:' after 'Thought:'

- Model chỉ viết thường “action:” thay vì “Action:” – với con người đây là lỗi không đáng kể, nhưng parser dựa trên regex coi là sai hoàn toàn
- Không có cơ chế nào để hệ thống “sửa” hay “đoán ý” – cả vòng lặp dừng lại vì 1 ký tự
- Đây chính là loại lỗi mà function calling muốn loại bỏ: đưa hợp đồng ra khỏi “đoán chữ”, chuyển vào schema có thể validate máy móc

Function Calling Ra Đời: Cho Model Một “Hợp Đồng” Rõ Ràng

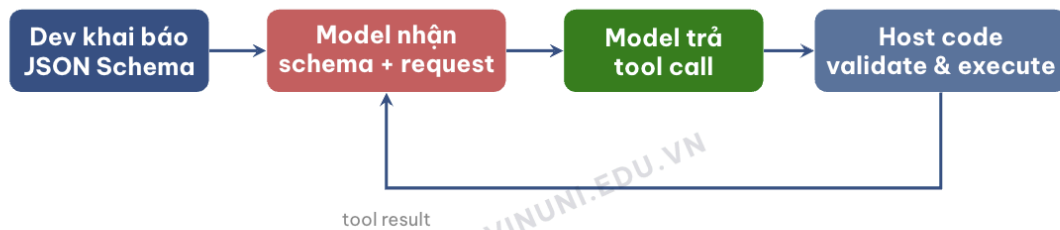
Text-based ReAct

- Action = chuỗi text tự do
- Hệ thống tự viết parser
- Không ai đảm bảo đúng định dạng

Function Calling

- Dev khai báo tool bằng **JSON Schema**: tên, mô tả, tham số
- Model trả về **structured object** khớp schema
- Không cần viết regex để đọc output

OpenAI ra mắt function calling tháng 6/2023 cho gpt-4-0613/gpt-3.5-turbo-0613; Anthropic tích hợp tool use tương tự cho Claude. Đây là bước chuyển từ “prompt model trả lời đúng format” sang “model được thiết kế để trả structured output”.



- Schema (tên, mô tả, tham số, kiểu dữ liệu) được gửi kèm mỗi request tới model
- Model không trả text tự do cho phần action – trả về một object có cấu trúc gồm name (tên tool) và arguments (tham số dạng JSON)
- Host code chỉ cần đọc field có sẵn, không cần đoán hay parse chuỗi

```
tools = [{
    "type": "function",
    "name": "search_flights",
    "description": "Find flights by route/date/budget",
    "parameters": {
        "type": "object",
        "properties": {"origin": {"type": "string"},
            "destination": {"type": "string"}},
        "required": ["origin", "destination"],
    },
}]

response = client.responses.create(
    model="gpt-4o", input=messages, tools=tools,
)

for item in response.output:
    if item.type == "function_call":
        print(item.name)      # "search_flights"
        print(item.arguments) # '{"origin": "HAN", ...}'
```

Model Có Cần Train Riêng Để Function Calling? – Có

- OpenAI xác nhận rõ: các model gpt-4-0613/gpt-3.5-turbo-0613 **đã được fine-tune** để (a) tự nhận biết khi nào cần gọi function, và (b) trả JSON đúng theo signature – đây không chỉ là prompting khéo léo
- Anthropic: với built-in tools, Claude được huấn luyện trên hàng ngàn trajectory dùng đúng schema đó, nên gọi tool đáng tin cậy và tự phục hồi lỗi tốt hơn so với tool tự định nghĩa
- Từ 2024, OpenAI thêm lớp thứ hai: **Structured Outputs** – schema được biên dịch thành grammar, ép decoding chỉ được sinh token hợp lệ với schema (không chỉ dựa vào việc model “muốn” tuân thủ)

OpenAI báo cáo: gpt-4o-2024-08-06 kèm Structured Outputs đạt **100%** tuân thủ schema trong eval nội bộ, so với dưới **40%** của gpt-4-0613 không có cơ chế này.

Lớp 1: Fine-tuning / RLHF

Model được huấn luyện thêm trên hàng ngàn trajectory “gọi tool đúng cách” – giúp model học được khi nào nên gọi tool nào, với tham số gì. Đây là xu hướng học được, không phải đảm bảo tuyệt đối.

Lớp 2: Grammar-constrained Decoding

JSON Schema được biên dịch thành grammar (finite-state machine); ở mỗi bước sinh token, chỉ token hợp lệ theo schema mới được phép chọn – đây là *đảm bảo* bằng kỹ thuật, không phụ thuộc model có “muốn” tuân thủ hay không.

Fine-tuning giúp model chọn đúng tool và đúng lý do (semantic); grammar constraint đảm bảo output luôn parse được (syntactic). Chỉ có lớp 1, model vẫn có thể lệch schema ở edge case hiếm gặp – lớp 2 mới là thứ đưa tỷ lệ tuân thủ lên gần như tuyệt đối.

Ưu Điểm Của Function Calling So Với Text-based ReAct

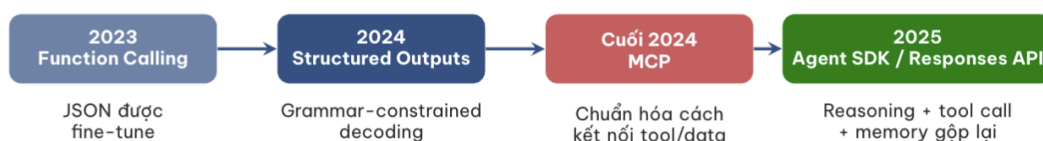
- ☒ **Schema validation gần như miễn phí:** grammar-constrained decoding khiến output sai schema gần như không thể xảy ra
- ☒ **Parallel tool calls:** model có thể trả nhiều tool call trong cùng một lượt, thay vì chỉ 1 action mỗi vòng như ReAct text
- ☒ **Không cần viết parser tay:** SDK trả object có field `.name` / `.arguments` sẵn, không cần regex
- ☒ **Độ tin cậy đo được:** có số liệu benchmark rõ ràng (100% so với dưới 40%) thay vì chỉ cảm tính “prompt tốt hơn”
- ☐ **Đánh đổi:** vẫn phụ thuộc chất lượng mô tả tool; tool mơ hồ vẫn khiến model chọn sai

So Sánh Chi Tiết: Text-based ReAct vs Function Calling

Khía cạnh	Text-based ReAct	Function Calling
Output format	Chuỗi text tự do	JSON object có cấu trúc theo schema
Validation	Không có – lỗi chỉ lộ ra lúc parse	Grammar-constrained decoding chặn ngay lúc sinh token
Cơ chế bảo đảm	Chỉ dựa vào prompting / in-context learning	Fine-tuning + constrained decoding
Error mode điển hình	<i>Could not parse LLM output</i>	Hiếm khi sai schema; lỗi còn lại là chọn sai tool
Parallel calls	Không – 1 action mỗi vòng lặp	Có – nhiều tool call trong 1 lượt
Công cụ cho dev	Regex/string parser tự viết, tự bảo trì	SDK trả object có field sẵn (.name, .arguments)
Độ tin cậy đo được	Không có số liệu chuẩn để so sánh	100% vs dưới 40% (OpenAI, có/không Structured Outputs)

So sánh đầy đủ hơn hai cơ chế trước khi sang phần agent hiện đại

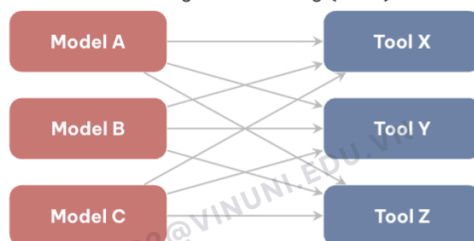
Từ Function Calling Đến Agent Như Hiện Nay (2025–2026)



- **MCP** (Anthropic, 11/2024): function calling chuẩn hóa cách gọi 1 tool, MCP chuẩn hóa cách tool/data được expose cho bất kỳ model/client – tránh tích hợp riêng cho từng cặp model-tool
- **Claude Agent SDK** (9/2025) và **OpenAI Responses API** (3/2025) đóng gói sẵn: reasoning loop + function calling + memory – vẫn là vòng lặp ReAct, nhưng Action giờ có train, có schema, chạy song song được

ReAct (Thought → Action → Observation) không biến mất – chỉ được nâng cấp hạ tầng bên dưới.

Trước MCP: mỗi cặp model-
tool cần glue code riêng ($M \times N$)



- Function calling chuẩn hóa **cách 1 model gọi 1 tool**, nhưng mỗi nhà cung cấp vẫn có thể khác định dạng – vẫn phải viết glue code cho từng cặp
- MCP (Anthropic, 11/2024) định nghĩa **giao thức chung**: server expose tool/data theo 1 chuẩn, client nào cũng kết nối theo đúng chuẩn đó – giảm $M \times N$ tích hợp xuống còn $M+N$

Lưu ý: Buổi Multi-agent / MCP / A2A sẽ đào sâu kiến trúc này; ở đây chỉ cần hiểu MCP giải quyết vấn đề gì.

1. **LLM thuần:** chỉ next-token prediction, không hành động được Kéo đến trang này để mở note riêng
2. **Prompt engineering:** few-shot/CoT giúp reasoning tốt hơn, nhưng vẫn 1 lượt, không gọi tool
3. **Text-based ReAct (2022):** thêm Action dạng text, nhưng phụ thuộc parser tự viết – dễ vỡ
4. **Function calling (2023+):** Action trở thành JSON schema được train + ép bằng constrained decoding – đáng tin cậy hơn hẳn
5. **Agent hiện nay:** MCP chuẩn hóa kết nối tool/data, Agent SDK/Responses API gộp reasoning loop + function calling + memory thành 1 primitive

Mỗi bước giải quyết đúng 1 giới hạn của bước trước – hiểu rõ giới hạn thì hiểu được vì sao ngành thiết kế ra bước tiếp theo, thay vì học thuộc tên công nghệ.

Agent Loop: Code Anatomy

Từ prompt, tool registry, đến loop control và framework hóa

```
messages = []

for step in range(MAX_ITERATIONS):
    output = call_model(
        system=SYSTEM_PROMPT,
        messages=messages,
        tools=TOOLS,
    )
    if output.type == "final_answer":
        return output.content

    result = run_tool(output.name, output.args)
    messages += [
        output.as_message(),
        tool_message(output.name, result),
    ]

return "Stopped: max iterations reached"
```

```
SYSTEM_PROMPT = """
You are a travel planning agent.

Your job:
- Break the user goal into smaller steps
- Use tools when fresh information is required
- Think briefly, then choose the best next action
- Stop when you have enough evidence to answer

Rules:
- Never invent tool results
- If a tool fails, explain the failure and try a fallback
- Keep internal thoughts short and actionable
- Output either a tool call or a final answer
"""
```

```
TOOLS = {  
  "get_weather": {  
    "description": "Weather by city/date",  
    "args": ["city", "date"],  
  },  
  "search_flights": {  
    "description": "Flights by route/date/budget",  
    "args": ["origin", "destination", "date", "max_price"],  
  },  
}
```

Max Iterations Safeguard: Tránh Agent Đi Vòng

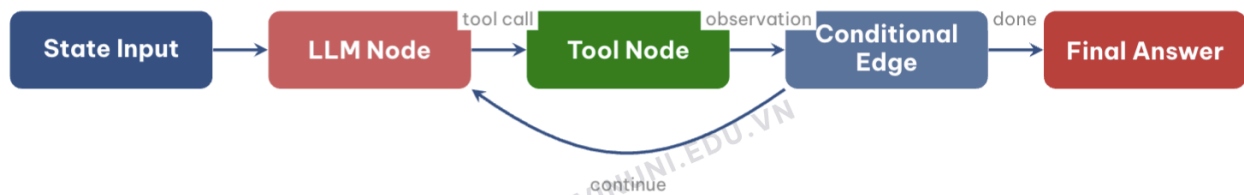
Cần guardrails gì?

- Giới hạn số vòng lặp
- Timeout cho từng tool
- Budget token / cost trần
- Retry có kiểm soát
- Fallback sang human hoặc chatbot

Dấu hiệu loop

- lặp lại cùng một tool call
- hỏi lại thông tin đã có
- reasoning không tiến thêm
- observation không thay đổi nhưng vẫn tiếp tục

Khi output không tiến triển, cùng một tool bị gọi lặp lại, hoặc observation không đổi mà agent vẫn tiếp tục, cần dừng loop và fallback.



- ReAct loop bằng tay phù hợp để học bản chất
- LangGraph giúp biểu diễn **state, nodes, edges, conditional routing** rõ hơn
- Khi workflow nhiều nhánh hoặc cần persist state, graph approach dễ maintain hơn loop ad-hoc

Live Demo & Debug

Build agent tra cứu thời tiết và gợi ý trang phục ngay trên lớp

1. Định nghĩa 2 tools: `get_weather` và `recommend_outfit`
2. Viết system prompt: agent chỉ được kết luận khi đã có dữ liệu thời tiết
3. Chạy loop và đọc trace Thought / Action / Observation
4. Cố tình tạo lỗi: tool timeout hoặc agent chọn sai outfit
5. Debug: sửa prompt, sửa tool description, hoặc thêm safeguard

Cho học viên thấy agent fail ở đâu và vì sao trace lại quan trọng hơn một final answer “trông có vẻ đúng”.

```
def get_weather(city: str, date: str) -> dict:
    return {
        "city": city,
        "date": date,
        "temperature_c": [27, 32],
        "rain_probability": 0.7,
    }

def recommend_outfit(temp_high: int, rain_probability: float) -> str:
    if rain_probability > 0.5:
        return "Ao mong, giày de kho, mang theo o gap."
    if temp_high > 30:
        return "Ao nhe, thoang, uu tien vai cotton."
    return "Trang phục thoải mái, có thể mang ao khoác nhe."
```

Nhìn vào trace trước

- Thought có đúng mục tiêu không?
- Agent chọn đúng tool chưa?
- Args truyền vào có hợp lệ không?
- Observation có bị thiếu field quan trọng không?

4 nơi thường phải sửa

- Tool description quá mơ hồ
- System prompt thiếu rule dừng
- Không có safeguard cho retry / loop
- Evaluation chỉ chấm final answer, không chấm trace

Lưu ý: Agent debugging gần với debugging distributed system hơn là chỉ prompt tuning. Ta phải nhìn cả model, tool, state, và orchestration.

Chatbot vs Agent

Khi nào mỗi loại thắng và tại sao hybrid pattern thường thực dụng nhất

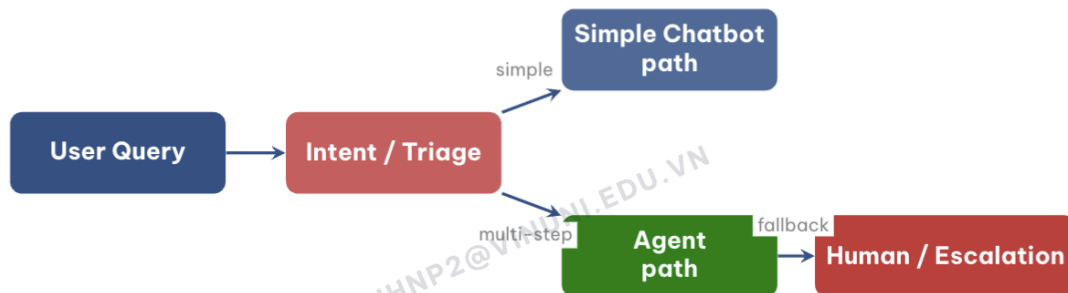
Khi Nào Chatbot Thắng, Khi Nào Agent Thắng?

Kéo đến trang này để mở note riêng

Khía cạnh	Chatbot thắng	Agent thắng
Tác vụ	FAQ, support đơn giản, nội dung 1 lượt	Booking, research, coding, data analysis nhiều bước
Tốc độ	Nhanh, ít round-trip	Chậm hơn do loop và tool calls
Cost	Thấp hơn, predictable hơn	Cao hơn nhưng đổi lại xử lý được bài toán khó hơn
Kiểm soát	Dễ hơn, ít state	Khó hơn vì cần orchestration và eval theo trace
UX	Phản hồi nhanh, đơn giản	Tạo cảm giác “làm việc giúp bạn” nếu làm tốt

Bắt đầu bằng chatbot là lựa chọn mặc định tốt

Hybrid Pattern: Thực Dụng Hơn Cực Đoan



Không cần chọn một phe. Thiết kế tốt thường là: triage nhanh, câu đơn giản đi chatbot path, câu phức tạp mới mở agent loop.

Thực Hành

Lab 3: Chatbot vs Agent – Hands-on Comparison

Cách Chạy Lab 3



1. Chọn lại use case từ Ngày 2 hoặc một use case tương đương
2. Build **chatbot baseline** cho bài toán đó
3. Nâng cấp thành **ReAct agent** có ít nhất 1–2 tools
4. Chạy **5 test cases** giống nhau trên cả hai hệ thống
5. Vẽ **flowchart** và ghi nhận nơi agent thực sự tạo thêm giá trị

Nhờ AI generate scaffolding code, nhưng nhóm phải tự sửa system prompt, tool description, và điều kiện dừng.

Mục tiêu: Build chatbot baseline rồi nâng cấp thành ReAct agent cho cùng một use case để so sánh trực tiếp

Deliverable: **Nộp cuối buổi:** chatbot + agent + 5 test cases + 1 trace + 1 flowchart

Bonus: thêm fallback path hoặc human escalation

Thời gian: 150 phút

Tổng Kết – Key Takeaways

- 1 Agent không phải “chatbot thông minh hơn”; agent = **LLM + reasoning + tools + memory/state**
- 2 ReAct là pattern dễ học nhất để biến LLM thành hệ thống biết hành động và dễ debug
- 3 Chỉ dùng agent khi bài toán có **multi-step reasoning, tool use, dynamic decisions, long horizon**
- 4 Trong production, guardrails, trace, và evaluation quan trọng không kém model quality

Prompt Engineering & Tool Calling

“Ngày mai ta đi sâu hơn vào cách viết `system prompt` production-grade và mô tả tools để agent dùng đúng ý.”

- Đọc lại trace lab hôm nay và tìm 1 chỗ agent ra quyết định chưa tối ưu
- Thử viết lại tool description theo hướng rõ input, output, và failure mode hơn

Tài Liệu Tham Khảo

- 1 Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, 2023.
- 2 Anthropic. *Building effective agents*. anthropic.com/research/building-effective-agents
- 3 LangChain / LangGraph docs. *Quickstart and Introduction*. langchain-ai.github.io/langgraph